

From Workbench to Factory: Agent Harnesses and a Persistent Agent Platform

Avinash Pyyety · White paper v0.1 · September 2026 · Writing assistance: SpaceXAI

The argument in one paragraph. Many enterprises now package engineering work as skills for IDE-based coding assistants. The next step is agents that work for longer, with less supervision, and across more of the enterprise's systems. An IDE can't provide that on its own. It is often hosted in snapshot virtual machines that don't persist, and its agent loop was built around a human at the keyboard. This paper recommends a specific platform layer under the IDE: persistent agent workspaces that run a *portfolio* of model-native harnesses (Claude Code, OpenAI Codex, xAI Grok Build), with one model-agnostic harness (OpenCode) as the common fallback. These harnesses share one skills and knowledge layer, one gateway to enterprise systems, and one path for metering tokens and auditing actions. The IDE stays the human cockpit, not the agents' runtime.

1. Context: the IDE-assistant starting point

A common first step in enterprise agentic engineering is to standardize on an IDE-based assistant, such as GitHub Copilot in VS Code. Engineering activities are packaged as reusable skills. IDE extensions give agents a local loop with few calls to production systems, which stay the system of record. Work state and notes are saved to shared drives so context carries across sessions. The most valuable asset built this way is usually a per-application knowledge graph covering lineage, contracts, job topology and entities, which agents need for triage, refactors and spec-driven builds.

That approach keeps a human in the loop, uses tooling that is often already approved, and proves the skill and knowledge model. The next question is which foundational unlocks agents need to work with greater autonomy. Harnesses are the first. This paper sets out the problem, the reasoning, and a specific platform to build on.

2. Problem statement

Greater autonomy means agents that plan, execute and verify multi-step work over hours, coordinate with other agents, and act on enterprise systems such as schedulers, IT service management, data warehouses, transformation tools, observability and data catalogs. Five limits of an IDE-only setup block this.

Limit	What typically happens	Why it blocks autonomy
1. The runtime isn't persistent	IDEs often run in snapshot or virtual desktop VMs. When the session ends, processes, caches, local indexes and in-progress state are gone.	An agent can't run overnight, resume a long job, keep a warm index of the codebase, or hold a working environment between tasks. Every session starts cold.
2. Round trips to	The IDE's agent loop is designed for a person	Autonomous work needs many cheap act-and-verify

enterprise systems are limited	reviewing each step. System access goes through extensions and tool calls never meant for long unattended chains.	cycles against real systems: run a job, read the logs, check quality results, fix, re-run. The IDE caps how far that loop can go.
3. Token efficiency	Tokens are consumed through a human-oriented assistant, with context rebuilt each session and little control over model choice, caching or routing.	Agent work needs cheap, fast tokens tuned for machines rather than people: prompt caching, routing small steps to smaller models, and compacting context. Without this, long runs cost too much.
4. Long-context work	Context is limited by the session and the IDE's context handling. Notes on shared drives help but aren't managed by the agent runtime itself.	Migrations, system-wide refactors and multi-day investigations need context that is managed deliberately: summarizing, checkpointing and retrieving from the knowledge graph.
5. Multi-agent orchestration	There is effectively one agent per IDE session.	Autonomous delivery needs parallel agents (build, test, quality, documentation) with handoffs, shared state and a coordinator. The IDE isn't built to host that.

An IDE assistant is itself a harness: an IDE with agent capabilities attached. The issue isn't that it's a poor tool. It is optimized for a human in the loop, and autonomy needs a runtime optimized for the agent.

3. Why harnesses matter, and why more than one

3.1 What a harness is

A harness is the software loop around a model. It decides what goes into context, which tools the model can call and how, how file edits and shell commands run, how results come back, how long runs are compacted, and when to stop or ask for help. With the same model, a different harness can produce very different results.

3.2 Harnesses are co-trained with their models

Frontier labs now post-train their models on their own harness. Supervised fine-tuning and reinforcement learning happen on the same tool schemas, edit formats and agent protocol the harness exposes. The result is a tight fit, so each lab's model performs best in the harness it was trained in:

- **Claude models** in **Claude Code** (Anthropic)
- **GPT models** in **Codex** (OpenAI)
- **Grok models** in **Grok Build** (xAI)

So harnesses and models can't be freely mixed without cost. Running a model outside its native harness usually works, but often with lower efficiency and reliability: more failed tool calls, worse edits and more tokens per completed task. This is a working hypothesis drawn from vendor positioning and hands-on use. The pilot in

Section 6 is designed to measure it on real tasks.

3.3 Why one harness isn't enough

No single model family leads on every task type, and leadership changes often, sometimes within weeks. Standardizing on one harness locks you into one lab's model roadmap. Standardizing on the IDE locks you into a human-paced loop.

3.4 Where a model-agnostic harness fits

Open, model-agnostic harnesses such as **OpenCode** can drive most models through one interface. They give a consistent baseline, easy model switching, a fallback when a vendor harness is unavailable, and a way to run open-weight or self-hosted models. The trade-off is that a general harness is unlikely to match each native harness on its own model. OpenCode is best used as the *baseline and fallback*, not the only harness. Other options, including IDE assistants' own agent and command-line modes, are worth tracking as they mature.

4. The specific platform: what sits under the harnesses

Harnesses on their own are only operational tooling. Autonomy comes from the platform they run on. Six specific components, each with a clear job:

Component	What it is	What it unlocks
A. Persistent agent workspaces	Long-lived cloud development machines per agent or team (container or VM), with persistent disk, warm dependencies, local code indexes and scheduled runs. These replace snapshot VMs as the agents' runtime.	Overnight and multi-day runs; resume after interruption; the agents' "own computer."
B. Harness layer	Claude Code, Codex and Grok Build installed in each workspace in headless or command-line mode, with OpenCode as baseline and fallback. Selected per task through a simple routing policy.	Each model runs in its native harness; no lock-in to one vendor.
C. Model and token gateway	One endpoint for all model traffic, providing authentication, per-team budgets, prompt caching, routing to smaller models for small steps, and cost metering per task.	Cheap, fast tokens for machines, with cost visibility and control.
D. Shared skills and knowledge layer	Engineering skills and per-application knowledge graphs, exposed through the Model Context Protocol (MCP), an open standard every major harness supports.	Build a skill once and use it in every harness; existing skill work carries forward instead of being rebuilt per vendor.
E. System access broker	A governed gateway to enterprise systems using scoped service identities, read-first defaults, approval gates for writes,	Many act-and-verify cycles against real systems, safely. Production systems

and rate limits.

stay the system of record.

F. Orchestration, memory and audit	A coordinator for multi-agent runs (task queue, handoffs, shared state), durable memory and checkpoints, and full traces of every prompt, tool call and system action.	Parallel agents; explainability, replay and compliance evidence.
------------------------------------	--	--

The IDE's new role. The IDE assistant stays the place where engineers review, steer and approve. Engineers connect to the persistent workspaces from the IDE, watch agent runs, and step in at approval gates. The investment in IDE skills and tooling is kept. It moves to supervising agents rather than being their only runtime.

5. Options considered

Option	Strengths	Weaknesses	View
1. Stay IDE-assistant only	Often already approved; no new vendors; familiar to engineers.	Doesn't fix the non-persistent runtime, the limits on long unattended loops or token control. Autonomy is capped.	Keep as the cockpit. Not enough as the runtime.
2. Standardize on one model-agnostic harness (OpenCode)	One tool to secure and support; model choice stays open.	Likely less efficient than each native harness on its own model; depends on community support.	Adopt as baseline and fallback.
3. Standardize on one vendor harness	Best fit for that one model; simpler contract.	Locked into one lab's roadmap; loses whenever another model leads a task type.	Not recommended.
4. Multi-harness portfolio on a shared platform	Each model runs where it performs best; skills and governance are shared; vendors can be swapped as the market moves.	More harnesses to secure and license; needs the platform components in Section 4.	Recommended , subject to a measured pilot.

6. How to prove it: a measured pilot

Run a time-boxed pilot on real engineering work before any broad rollout.

- **Task set:** 20 to 30 representative tasks, for example data quality triage from an observability alert, a transformation model refactor, a scheduler job failure investigation, spec-to-pipeline build with tests, and lineage-driven impact analysis.
- **Contenders:** the current IDE assistant (baseline), Claude Code, Codex, Grok Build and OpenCode. Every contender uses the same MCP skills and the same sandbox.
- **Metrics per task:** completed correctly (yes or no, with human review), tokens and cost per completed task, wall-clock time, system round trips, human interventions, and unattended run length before the

agent needs help.

- **Decision rule:** keep a harness in the portfolio only where it clearly beats the baseline on the task types it's routed to. Otherwise, route that work to OpenCode or the IDE assistant.

Numbers should come only from the pilot; this paper claims none in advance.

7. Risks and how to manage them

Risk	Mitigation
Agents take harmful actions on production systems	Read-first access; writes pass approval gates; scoped service identities; sandbox first; full audit trail.
Data leaves the organization's boundary	All model traffic goes through the gateway under approved data-processing terms; no direct consumer endpoints.
Token costs grow out of control	Per-team budgets, cost metering per task, caching and routing to smaller models; the pilot measures cost per completed task.
Too many tools to support	Shared skills through MCP and one platform layer, so a harness is a replaceable component; prune the portfolio on pilot results.
Vendor harnesses change quickly	OpenCode fallback; no skills built for one vendor only; quarterly re-evaluation.